

## MySQL Stored Procedure Variables

A **variable** is a named data object whose value can change during the stored procedure execution. You typically use variables in stored procedures to hold immediate results. These variables are local to the stored procedure.

Before using a variable, you must declare it.

### Declaring variables

To declare a variable inside a stored procedure, you use the DECLARE statement as follows:

```
DECLARE variable_name datatype(size) [DEFAULT default_value];
```

#### Examples:

```
DECLARE totalSale DECIMAL(10,2) DEFAULT 0.0;
```

```
DECLARE x, y INT DEFAULT 0;
```

Note: If you declare a variable without specifying a default value, its value is NULL.

### Assigning variables

Once a variable is declared, it is ready to use. To assign a variable a value, you use the SET statement:

```
SET variable_name = value;
```

#### Example:

```
DECLARE total INT DEFAULT 0;  
SET total = 10;
```

### Variable scopes

A variable has its own scope that defines its lifetime. If you declare a variable inside a stored procedure, it will be out of scope when the END statement of stored procedure reaches.

When you declare a variable inside the block BEGIN END, it will be out of scope if the END is reached.

MySQL allows you to declare two or more variables that share the same name in different scopes. Because a variable is only effective in its scope. However, declaring variables with the same name in different scopes is not good programming practice.

A variable whose name begins with the @ sign is a session variable. It is available and accessible until the session ends.

### MySQL SELECT INTO Variable

The MySQL SELECT INTO variable is used to store query result in variables.

To store query result in one or more variables, you use the SELECT INTO variable syntax:

```
SELECT
    col1, col2, col3, ...
INTO
    @var1, @var2, @var3,...
FROM
    table_name
WHERE
    condition;
```

- col1, col2, and col3 are columns or expressions that you want to select and store into the variables.
- @var1, @var2, and @var3 are the variables which store the values from col1, col2 and col3.
- The number of variables must be the same as the number of columns or expressions in the select list. In addition, the query must return zero or one row.
- If the query return no rows, MySQL issues a warning of no data and the value of the variables remain unchanged.
- In case the query returns multiple rows, MySQL issues an error. To ensure that the query always returns maximum one row, you use the LIMIT 1 clause to limit the result set to a single row.

#### Example:

```
select custAddress
into @address
from customer
where custID='101';
```

- *The statement above gets the address of the customer whose customer ID is '101' and stores it in the @address variable.*

The following statement displays the content of the @address variable

```
select @address;
```

### Putting them all together

The following example illustrates how to use SELECT INTO, and how to declare and use a variable in a stored procedure:

```
DELIMITER $$

CREATE PROCEDURE GetTotalOrder(productID char (3))
BEGIN
    DECLARE totalOrder INT DEFAULT 0;

    SELECT sum(qtyOrdered)
    INTO totalOrder
    FROM orderdetail
    WHERE prodID=productID;

    SELECT totalOrder;
END$$

DELIMITER ;
```